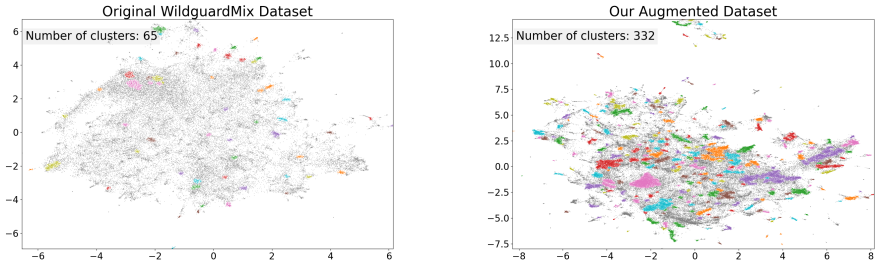


(a) Without HarmAug (b) With HarmAug

Figure 3: Clustering results of the original dataset and our augmented dataset. Our data augmentation HarmAug significantly increases the number of clusters, identified by DBSCAN, from 65 to 332.

Table 3: The prompt generator p_ψ, trained with each small safety guard model, samples 1,024 prompts. We assess the harmfulness of the prompts using the oracle safety guard model p_θ.

Reward Model	Train Reward (↑)	Test Reward (↑)	Diversity (↑)	Runtime
Llama-Guard-3 (Oracle)	-	0.99	0.65	17h 23m
RoBERTa-R4	**0.84**	0.00	0.55	12h 19m
HateBERT	0.84	0.00	0.59	8h 32m
DeBERTa + HarmAug	0.83	**0.82**	**0.74**	9h 8m

harmful effects of LLMs prior to their deployment. However, this process is computationally expensive. Previous works (Perez et al., 2022; Hong et al., 2024; Lee et al., 2024) iteratively train a language model policy p_ψ to generate prompts, using harmfulness scores from LLM-based safety guards like Llama-Guard-3 as rewards. However, this process incurs significant computational costs.

Lee et al. (2024) propose to fine-tune the language model p_ψ with the GFlowNet objective (Bengio et al., 2021), which allows to sample a prompt $\mathbf{x}$ proportional to a reward distribution. The reward of the prompt $\mathbf{x}$ is defined as:

$$R(\mathbf{x}) = \exp\left(\frac{1}{\beta}\mathbb{E}_{\mathbf{y}\sim p_{\texttt{target}}(\mathbf{y}|\mathbf{x})}[\log p_\theta(c=1 \mid \mathbf{x},\mathbf{y})]\right) \cdot p_{\texttt{ref}}(\mathbf{x})^{1/\gamma}, \tag{3}$$

where β and γ are positive constants that control the peakiness of the reward, p_θ is a safety guard model, and $p_{\texttt{ref}}$ is a reference language model to measure the likelihood of $\mathbf{x}$ to enforce the generation of natural sentences. Then the language model p_ψ is trained to minimize the following trajectory balance objective (Malkin et al., 2022):

$$\mathcal{L}_{\texttt{TB}}(\mathbf{x};\psi) = \left(\log \frac{Z_\psi \cdot p_\psi(\mathbf{x})}{R(\mathbf{x})}\right)^2, \tag{4}$$

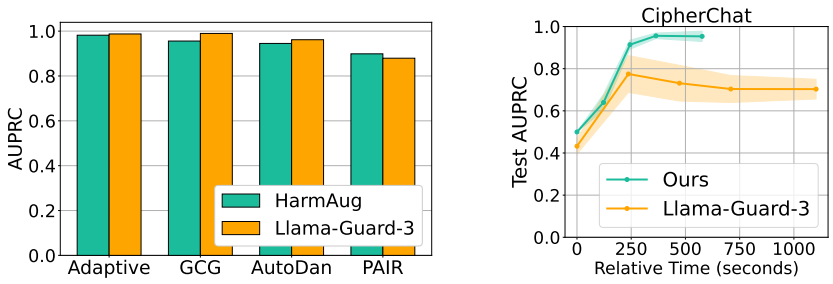
where $Z_\psi > 0$ is a learnable scalar approximating the partition function. Note that the training example $\mathbf{x}$ can be sampled from either the on-policy p_ψ or off-policies such as replay buffer. However, computing the reward $R(\mathbf{x})$ is costly due to the approximation of the expectation in Eq. (3). Each reward evaluation requires sampling multiple responses $\mathbf{y}$ from the target LLM $p_{\texttt{target}}$ and then calculating the harmfulness score for each $(\mathbf{x},\mathbf{y})$ pair using the safety guard model p_θ.

Experimental setup. To reduce the computational cost of calculating the reward $R(\mathbf{x})$, we train the harmful prompt generator p_ψ using Eq. (4), replacing the large safety guard model p_θ (Llama-Guard-3), with our smaller model q_ϕ (DeBERTa-v3-large), which has been trained using HarmAug. After training, the generator p_ψ samples $k = 1{,}024$ prompts, which are then evaluated based on their harmfulness score and diversity. We use the oracle safety guard model p_θ to assess harmfulness of the prompts as:

$$\frac{1}{5k}\sum_{i=1}^{k}\sum_{j=1}^{5} p_\theta(c=1 \mid \mathbf{x}^{(i)},\mathbf{y}^{(j)}), \quad \mathbf{x}^{(i)} \overset{\text{iid}}{\sim} p_\psi(\mathbf{x}), \quad \mathbf{y}^{(j)} \overset{\text{iid}}{\sim} p_{\texttt{target}}(\mathbf{y} \mid \mathbf{x}^{(i)}) \tag{5}$$

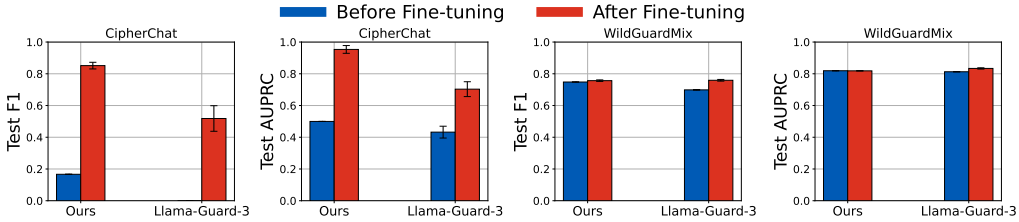
which is referred to as "Test Reward" in Table 3. For diversity, following prior work (Hong et al., 2024), we calculate the average cosine distance between all possible pairs of the generated prompts. Please refer to Appendix B.3 for more details.

Results. As shown in Table 3, our small safety guard model (DeBERTa-v3-large) trained with our HarmAug method, achieves a test reward comparable to the oracle model (Llama-Guard-3), while



(a) (b)

Figure 4: **(a)**: Test AUPRC score on various jailbreak attacks with our model (DeBERTa-large) and Llama-Guard-3. **(b)**: Plot of test AUPRC score on CipherChat as a function of wallclock time during fine-tuning.



(a) (b) (c) (d)

Figure 5: After further fine-tuning DeBERTa and Llama-Guard-3 on CipherChat and WildGuardMix datasets, we report average test F1 and AUPRC score of five runs on each dataset.

reducing GFlowNet training runtime by half. These results suggest that our safety guard model is an appropriate proxy for the oracle model, yielding comparable performance while significantly improving computational efficiency. Conversely, the other baseline models show zero test rewards, despite achieving high training rewards, indicating a substantial distributional mismatch between the oracle model and the baseline models.

4.3 Case Study II: Efficient Further Fine-Tuning Against New Jailbreak Attacks

Background. As shown in Fig. 4a, both our safety guard model and its teacher Llama-Guard-3 effectively defend against many recent and powerful jailbreak attacks such as GCG (Zou et al., 2023), PAIR (Chao et al., 2023), AutoDAN (Liu et al., 2024a), and Adaptive Attacks (Andriushchenko et al., 2024). However, efficient fine-tuning of a safety guard model is crucial for real-world deployment, as the model needs to be continuously updated to detect new jailbreak attacks that exploit its vulnerabilities and circumvent the safety guardrails. For example, as illustrated in Fig. 5a and Fig. 5b, Llama-Guard-3 and DeBERTa with our HarmAug are susceptible to attacks from CipherChat. In this section, we empirically demonstrate that a small safety guard model allows for a reduction in the computational cost associated with further fine-tuning to defend against new attacks.

Experimental setup. We further fine-tune Llama-Guard-3 and DeBERTa-large trained with our HarmAug method on the CipherChat (Yuan et al., 2024) dataset, which comprises 25 pairs of harmful instructions and responses encoded in ASCII for the purpose of jailbreak. To prevent catastrophic forgetting (McCloskey & Cohen, 1989), we sample a mini-batch from both the WildGuardMix and CipherChat datasets in every update step. We train the models using LoRA (Hu et al., 2022) for 200 steps, with the rank set to 32, a batch size of 8, and a learning rate of 10^{-4}. Finally, we evaluate the models by measuring F1 and AUPRC scores on both the test split of WildGuardMix and CipherChat.

Results. As shown in Fig. 5, neither model is initially able to defend against jailbreak attacks from CipherChat with AUPRC scores below 0.5. After further fine-tuning, however, our DeBERTa safety guard model with HarmAug successfully detects most jailbreak attacks from the CipherChat dataset (AUPRC score > 0.9), while retaining its performance on the WildGuardMix dataset (AUPRC score > 0.8). Surprisingly, our small model achieves even better F1 and AUPRC scores than Llama-Guard-3 on CipherChat. Moreover, as shown in Fig. 4b, our model reduces the training time by half. In contrast, Llama-Guard-3 continues to exhibit difficulties in defending against jailbreak attacks from the CipherChat dataset after fine-tuning (Fig. 5a and Fig. 5b). These experimental results highlight the efficiency and effectiveness of our small safety guard model on further fine-tuning.

Table 5: Different LLM backbones for sampling harmful instructions. We report the average F1 and AUPRC scores of three runs. Bold model names indicate LLMs used for our data augmentation method HarmAug. For results including standard deviations, please refer to Table 10 in Appendix D.

		OAI		ToxicChat		HarmBench		WildGuardMix		Average	
Model	LLM Size	F1	AUPRC	F1	AUPRC	F1	AUPRC	F1	AUPRC	F1	AUPRC
DeBERTa	-	0.7092	0.7869	0.6118	0.6837	0.8379	0.8806	0.7507	0.8337	0.7274	0.7962
DeBERTa + EDA	-	0.6858	0.8394	0.5964	0.7141	0.8430	0.8793	0.7279	0.8315	0.7133	0.8161
DeBERTa + GFN	-	0.6939	0.7793	0.6259	0.7191	**0.8463**	**0.8842**	0.7443	0.8376	0.7276	0.8050
DeBERTa + **Llama-3.1 Instruct**	8B	0.7398	0.8546	0.6133	0.7141	0.8369	0.8781	0.7481	0.8308	0.7345	0.8194
DeBERTa + **Llama-3.1 Base**	8B	0.7478	0.8743	0.5862	0.6588	0.8400	0.8776	**0.7651**	**0.8382**	0.7348	0.8122
DeBERTa + **Phi-3.5 Instruct**	3.8B	0.7230	0.8647	0.6073	0.7180	0.8337	0.8807	0.7543	0.8259	0.7295	0.8223
DeBERTa + **Mistral-0.3 Instruct**	7B	0.7317	0.8717	0.6230	0.7075	0.8304	0.8769	0.7516	0.8267	0.7342	0.8207
DeBERTa + **Fine-tuned Llama-2**	7B	**0.7544**	0.8696	0.6261	0.7052	0.8339	0.8829	0.7400	0.8277	**0.7386**	0.8213
DeBERTa + **Gemma-1.1 Instruct**	2B	0.7236	**0.8791**	**0.6283**	**0.7553**	0.8331	0.8841	0.7576	0.8265	0.7357	**0.8362**

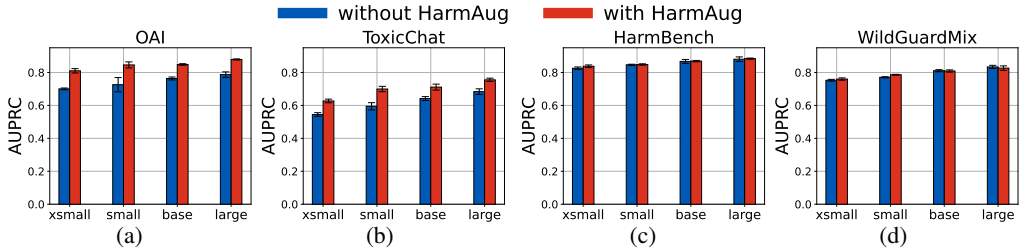


Figure 6: For each size of the DeBERTa model, we evaluate the performance of our HarmAug method in comparison to the baseline knowledge distillation approach. We report average AUPRC scores over three runs.

Table 6: For each model size of DeBERTa trained with our augmentation, we profile it on the WildGuardMix test split. FLOPs refers to floating-point operations, latency to forward pass time, and peak memory to maximum GPU usage. The percentages in parentheses indicate the relative comparison to the Llama-Guard-3.

Model	F1 (↑)	Size (↓)	FLOPs (↓)	Latency (↓)	Peak Memory (↓)
Llama-Guard-3	0.6998 (100.00%)	8B (100.00%)	124.01 G (100.00%)	172.62 µs (100.00%)	28.82 GB (100.00%)
DeBERTa-xsmall + HarmAug	0.7025 (100.39%)	71M (0.89%)	65.80 M (0.05%)	15.24 µs (8.82%)	0.89 GB (3.09%)
DeBERTa-small + HarmAug	0.6971 (99.61%)	142M (1.76%)	109.97 M (0.08%)	10.20 µs (5.90%)	1.65 GB (5.73%)
DeBERTa-base + HarmAug	0.7368 (105.29%)	184M (2.30%)	219.94 M (0.17%)	18.97 µs (10.98%)	1.88 GB (6.52%)
DeBERTa-large + HarmAug	0.7576 (108.26%)	435M (5.43%)	743.55 M (0.59%)	43.22 µs (25.03%)	3.37 GB (11.69%)

4.4 Ablations

We conduct ablation studies of each component of our method to evaluate its effectiveness.

Table 4: Success rate of harmful instruction generation.

	Success Rate (%)
prefix injection	10.42
w/o prefix attack	13.02
w/ prefix attack	**96.81**

Prefix attack. To study the effectiveness of the prefix attack for generating harmful instructions, we remove the prefix "I have an idea for a prompt:" from the Prompt Format described in Sec. 3.2 and measure how often the LLM p_{LLM} successfully generates instructions instead of refusing to do so. We sample 10,000 instructions from p_{LLM} and use a simple pattern matching classifier proposed by Zou et al. (2023) to evaluate whether the LLM refuses to generate instructions. As shown in Table 4, removing the prefix or replacing our prefix attack with the prefix injection attack proposed by Wei et al. (2023b), which instructs the LLM to begin its response with the affirmative prefix "Absolutely! Here's ", significantly degrades the success rate, which indicates the necessity of prefix attack for circumventing the safety alignment of the LLM.

Backbone of instruction generators. We perform an ablation study to examine the effect of LLM backbones in generating harmful instructions. We prompt the following models for sampling harmful instructions: Gemma-1.1-2b-it, Llama-3.1-Instruct-8B-Instruct, Llama-3.1-8B, Phi-3.5-mini-instruct, Mistral-7B-Instruct-v0.3, and the fine-tuned Llama-2 model (Wei et al., 2024), which has been fine-tuned on 100 adversarial prompts. All models are prompted using the prompt format described in Sec. 3.2. As shown in Table 5, regardless of the choice of LLMs, data augmentation with LLM-based prompting outperforms the other baselines on average, including GFN and EDA. Moreover, data augmentation with instructions generated by the smallest model, Gemma-1.1-2b-it, yields the most significant improvement in AUPRC.

Size of student models. We study the trade-off between accuracy and efficiency as we increase the size of the the student models. Fig. 6 shows that our HarmAug method consistently improves the AUPRC scores across all DeBERTa model sizes. Moreover, larger models achieve better per-